

Practical No. 8

Aim

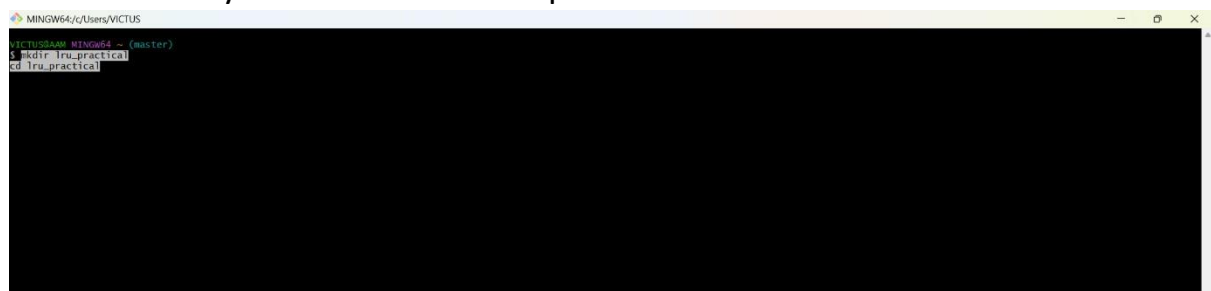
To write and execute a program to calculate **page hits and page faults** using **LRU (Least Recently Used) page replacement algorithm**.

Step 1: Open Git Bash

Git Bash was opened to execute commands and run the program.

Step 2: Create Directory

A new directory was created for the practical:

A screenshot of a Git Bash terminal window. The title bar shows 'MINGW64/c/Users/VICTUS'. The terminal content shows the user 'VICTUS@AAM MINGW64 ~ (master)' executing the commands 'mkdir lru_practical' and 'cd lru_practical'.

Step 3: Create C File A C file was created using nano editor

A screenshot of a Git Bash terminal window. The title bar shows 'MINGW64/c/Users/VICTUS/lru_practical'. The terminal content shows the user 'VICTUS@AAM MINGW64 ~ (master)' executing the commands 'mkdir lru_practical', 'cd lru_practical', and 'nano lru.c'.

Step 4: Write Program

The LRU Page Replacement program was written in C language with:

- Number of frames = 3
- Page reference string = 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5The program calculates total page hits and page faults.

```
MINGW64/c/Users/VICTUS/lru_practical
GNU nano 8.7 lru.c
#include <stdio.h>

int main() {
    int frames = 3;
    int pages[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    int n = 12;

    int memory[3];
    int time[3];
    int i, j;
    int hits = 0, faults = 0;
    int counter = 0;

    // initialize memory
    for(i = 0; i < frames; i++) {
        memory[i] = -1;
        time[i] = 0;
    }

    for(i = 0; i < n; i++) {
        int page = pages[i];
        int found = 0;

        // check hit
        for(j = 0; j < frames; j++) {
            if(memory[j] == page) {
                hits++;
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }

        // page fault
        if(!found) {
            int lru_index = 0;

            for(j = 1; j < frames; j++) {
                if(time[j] < time[lru_index]) {
                    lru_index = j;
                }
            }

            memory[lru_index] = page;
            counter++;
            time[lru_index] = counter;
            faults++;
        }
    }

    printf("Total Hits = %d\n", hits);
    printf("Total Faults = %d\n", faults);
}
```

```
MINGW64/c/Users/VICTUS/lru_practical
GNU nano 8.7 lru.c
int i, j;
int hits = 0, faults = 0;
int counter = 0;

// initialize memory
for(i = 0; i < frames; i++) {
    memory[i] = -1;
    time[i] = 0;
}

for(i = 0; i < n; i++) {
    int page = pages[i];
    int found = 0;

    // check hit
    for(j = 0; j < frames; j++) {
        if(memory[j] == page) {
            hits++;
            counter++;
            time[j] = counter;
            found = 1;
            break;
        }
    }

    // page fault
    if(!found) {
        int lru_index = 0;

        for(j = 1; j < frames; j++) {
            if(time[j] < time[lru_index]) {
                lru_index = j;
            }
        }

        memory[lru_index] = page;
        counter++;
        time[lru_index] = counter;
        faults++;
    }
}

printf("Total Hits = %d\n", hits);
printf("Total Faults = %d\n", faults);

float hit_ratio = (float)hits / n * 100;
float fault_ratio = (float)faults / n * 100;

printf("Hit Percentage = %.2f%%\n", hit_ratio);
printf("Fault Percentage = %.2f%%\n", fault_ratio);

return 0;
}
```

Step 5: Compile Program

The program was compiled using GCC compiler:

MINGW64/c/Users/VICTUS/lru_practical

```
VICTUS@AAM MINGW64 ~ (master)
$ mkdir lru_practical
$ cd lru_practical

VICTUS@AAM MINGW64 ~/lru_practical (master)
$ nano lru.c

VICTUS@AAM MINGW64 ~/lru_practical (master)
$ gcc lru.c -o lru
```

Step 6: Output Observed The output obtained is:

- Total Hits = 2
- Total Faults = 10
- Hit Percentage = 16.67%
- Fault Percentage = 83.33%

MINGW64/c/Users/VICTUS/lru_practical

```
VICTUS@AAM MINGW64 ~ (master)
$ mkdir lru_practical
$ cd lru_practical

VICTUS@AAM MINGW64 ~/lru_practical (master)
$ nano lru.c

VICTUS@AAM MINGW64 ~/lru_practical (master)
$ gcc lru.c -o lru

VICTUS@AAM MINGW64 ~/lru_practical (master)
$ ./lru
Total Hits = 2
Total Faults = 10
Hit Percentage = 16.67%
Fault Percentage = 83.33%

VICTUS@AAM MINGW64 ~/lru_practical (master)
$ |
```

The program for LRU page replacement algorithm was successfully executed, and the page hits and page faults were calculated correctly.